# REPORTE OFICIAL DEL PROYECTO: JUEGO UNO

*Materia: Interfaces Gráficas con Aplicaciones | Parcial 2*
*Arquitectura C# (Windows Forms) + API Python (FastAPI) + MySQL*

## 1. Descripción General del Proyecto

El presente proyecto consiste en el desarrollo e implementación del juego de mesa **UNO** adaptado para un equipo de 4 jugadores en pantalla, cumpliendo con las reglas oficiales del juego (validación de tiros por color o número, cartas de salto, reversa, +2, cambio de color y comodín +4). Para este segundo parcial, la aplicación se ejecuta localmente en una sola computadora donde una sola persona controla de forma interactiva los turnos de todos los participantes.

**Arquitectura del Sistema:** El programa no se conecta directamente a la base de datos desde C#. En su lugar, utiliza una arquitectura multinivel en la que la interfaz gráfica en C# (Windows Forms) se comunica mediante solicitudes HTTP GET y POST en formato JSON con una API cliente/servidor desarrollada en Python con FastAPI (ejecutada en el servidor mediante Uvicorn). La API procesa la lógica de persistencia, realiza las consultas mediante mysql-connector-python al gestor local MySQL Server / Workbench y devuelve los resultados serializados en JSON a la aplicación cliente.

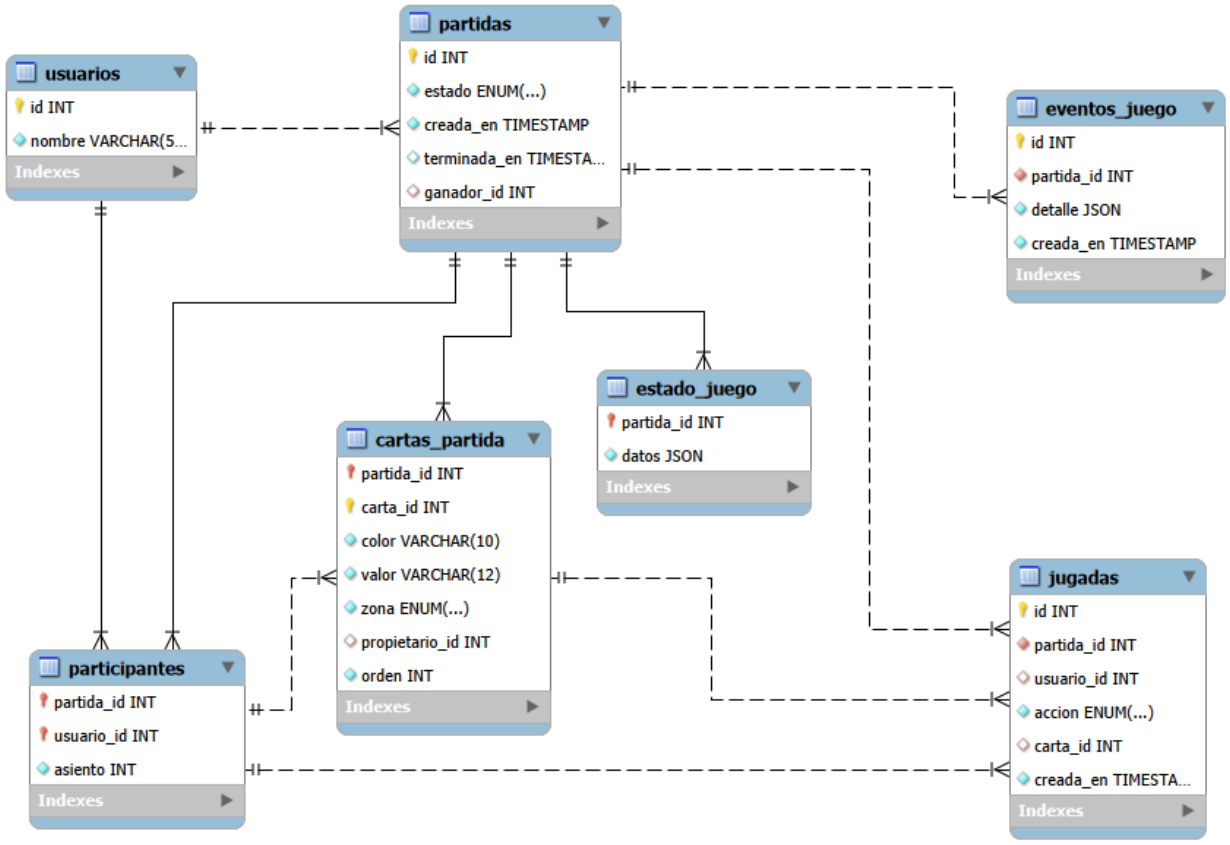
## 2. División de Tareas entre los Integrantes del Equipo

Para garantizar un desarrollo modular y cumplir con el control de versiones en GitHub, el trabajo se distribuyó formalmente entre los 4 integrantes del equipo:

| Integrante | Rol Asignado | Módulos y Entregables Específicos |
|---|---|---|
| Jacqueline Gómez<br><br>**Usuario de GitHub**: @Jacqueline0595 | Base de Datos & API Python | - Implementación de cambios en el diseño de las interfaces.<br>- Diseño del script SQL en MySQL Workbench, creación de las 7 tablas.<br>- Desarrollo de una API REST en Python con FastAPI y mysql.connector.<br>- Cambios en la parte visual de las funciones.<br>- Corrección de errores, código duplicado. |
| Alejandro Mora<br><br>**Usuario de GitHub**: @jandromt1 | Lógica del Juego (POO C#) | - Correcciones al diseño de las cartas y verificación del flujo del juego.<br>- Implementación de cartas especiales.<br>- Lógica de la carta +2 y optimización de la búsqueda. |

| | | |
|---|---|---|
| | | - Implementación de la carta reversa.<br>- Implementación de sistema de puntuación acumulativa por rondas según reglas oficiales |
| Aslan Cisneros<br><br>**Usuario de GitHub**: ZABNOOB | Interfaz Gráfica (GUI C#) | - Función de añadir carta.<br>- Diseño del formulario Form1.cs en Windows Forms,<br>- distribución espacial y función de repartir las manos de los 4 jugadores,<br>- mazo central, pozo de descarte<br>- e integración de imágenes de cartas. |
| Tito Durán<br><br>**Usuario de GitHub**: Axtraid | Integración C#-API & Git Admin | - Resuelve conflictos al integrar el main.<br>- Corrección de archivos al compilar.<br>- Conexión HTTP JSON entre C# y la API de Python.<br>- Rediseño del fondo de la mesa de juego.<br>- Rotación de perspectiva del jugador en el modo en red.<br>- Corrección de error de compilación en el servidor. |

# 3. Diagrama Entidad-Relación y Estructura de la Base de Datos Local




La base de datos se implementa de forma estrictamente local en MySQL Server. El modelo entidad-relación incluido contempla siete tablas para organizar los jugadores, las partidas, sus participantes, las cartas y los movimientos. Esta estructura cumple con los requisitos para identificar a los jugadores, consultar el historial de partidas y mantener un registro de las acciones realizadas en cada una.

1. **Tabla `usuarios`**
2. **Tabla `partidas`**
3. **Tabla `participantes`**
4. **Tabla `cartas_partida`**
5. **Tabla `estado_juego`**
6. **Tabla `eventos_juego`**
7. **Tabla `jugadas`**

# 4. Historial de Commits en Git y GitHub

De acuerdo con la indicación estricta del profesor, los 4 integrantes del equipo mantuvieron una actividad continua de commits registrados desde el 8 de noviembre de 2026 hasta la víspera de la entrega. A continuación se presenta la síntesis del registro del historial (`git log`):

| Fecha / Hora | Autor (Usuario GitHub) | Rama | Mensaje del Commit |
|---|---|---|---|
| Sep 21, 2026, 11:37 AM | @Jacqueline0595 | main | Initial commit |
| Sep 22, 2026, 2:14 PM | ZABNOOB | main | Inserción de la solución de la interfaz |
| Sep 22, 2026, 7:35 PM | @jandromt1 | main | Estructura inicial del proyecto de Windows Forms |
| Sep 22, 2026, 8:00 PM | @jandromt1 | feature/logica-e-interfaz | Agrega archivo .gitignore para proteger el proyecto |
| Sep 22, 2026, 11:57 PM | ZABNOOB | main | Primer demo de la interfaz. Añadir imagenes para las cartas del Juego |
| Sep 23, 2026, 12:18 AM | ZABNOOB | main | Solución de problema, rama no encontrada |
| Sep 24, 2026, 11:48 AM | ZABNOOB | main | Añadir carpeta de archivos de Interfaz UNO |
| Sep 24, 2026, 9:39 PM | ZABNOOB | main | Segundo intento de insertar carpeta Interfaz |
| Sep 29, 2026, 11:50 AM | @Jacqueline0595 | main | Prueba |
| Sep 29, 2026, 11:50 AM | ZABNOOB | main | Añadir función. Repartir cartas |
| Sep 29, 2026, 10:05 PM | ZABNOOB | main | Añadir características principales de la Interfaz, barajeo según las reglas del UNO |

| | | | |
|---|---|---|---|
| Sep 29, 2026, 10:06 PM | ZABNOOB | main | Merge branch 'main' of https://github.com/Jacqueline0595/JuegoUno |
| Oct 01, 2026, 11:30 AM | ZABNOOB | main | Se intercambia carpeta Interfaz por InterfazUno |
| Oct 01, 2026, 7:39 PM | @jandromt1 | main | Corrección de rotación visual para cartas lanzadas por jugadores laterales y verificación de flujo básico de juego |
| Oct 01, 2026, 7:52 PM | @jandromt1 | main | Implementación de la lógica para la carta especial de Bloqueo (Skip) |
| Oct 01, 2026, 8:12 PM | @jandromt1 | main | Implementacion de logica para la carta +2 y optimizacion del buscador dinamico de color y tipo |
| Oct 02, 2026, 8:32 AM | @Jacqueline0595 | main | Actualizar la interfaz integrando un icono representativo |
| Oct 02, 2026, 11:43 AM | @Jacqueline0595 | main | Commit para que se suba la imagen .ico |
| Oct 02, 2026, 11:50 AM | @Jacqueline0595 | main | Ahora si ya subi la imagen |
| Oct 03, 2026, 11:09 AM | @jandromt1 | main | Implementacion de la lógica para la carta especial de Reversa |
| Oct 03, 2026, 8:10 PM | @jandromt1 | main | Implementacion final de selector de color usando ComboBox e interfaz dinamica para Comodin de color y el +4 |
| Oct 03, 2026, 8:48 PM | @jandromt1 | main | Optimización visual de la mano de cartas e implementación de |

| | | | |
|---|---|---|---|
| | | | reorganización automática estilo abanico |
| Oct 03, 2026, 9:04 PM | @jandromt1 | main | Implementación de contenedor translucido (dorado) para resaltar dinámicamente la zona de cartas del jugador activo |
| Oct 03, 2026, 9:15 PM | @jandromt1 | main | Implementación de la regla oficial de avisar ¡UNO! y validaciones de castigo |
| Oct 03, 2026, 9:30 PM | @jandromt1 | main | Implementacion de lógica de fin de juego, pantalla de victoria y desactivación de mazo |
| Oct 03, 2026, 10:10 PM | @jandromt1 | main | Feat: Implementacion de sistema de puntuación acumulativa por rondas según reglas oficiales |
| Oct 05, 2026, 10:15 AM | @Jacquelin e0595 | main | Incorpore la barra superior y le quite la opción de maximizar o minimizar |
| Oct 05, 2026, 11:56 AM | Axtraid | tito | Mi parte, api y conexión entre códigos |
| Oct 05, 2026, 3:57 PM | @Jacquelin e0595 | jacque | Primeras implementaciondes del cambio de color |
| Oct 05, 2026, 7:39 PM | @Jacquelin e0595 | main | Merge branch 'main' into lógica-juego |
| Oct 05, 2026, 7:51 PM | @Jacquelin e0595 | main | Todo correcto |
| Oct 05, 2026, 8:07 PM | @Jacquelin e0595 | jacque | Uni lo que hizo alex con lo mio, aun no esta terminado |
| Oct 05, 2026, 9:35 PM | @Jacquelin e0595 | jacque | Se reparten las cartas a color y despues de escoger el turno, se ven en blanco y negro |

| | | | |
|---|---|---|---|
| Oct 06, 2026, 11:00 AM | @Jacquelin e0595 | jacque | Eliminacion de lineas innecesarias |
| Oct 06, 2026, 3:02 PM | @jandromt1 | feature/mejo rar-baraja | Se refactoriza Baraja_Click para dar oportunidad de usar carta que se toma del mazo |
| Oct 06, 2026, 11:52 PM | ZABNOOB | main | Merge manual de la union de base de datos y lógica con la interfaz del juego |
| Oct 07, 2026, 12:19 AM | ZABNOOB | main | Corrección de merge de github |
| Oct 09, 2026, 11:08 PM | @Jacquelin e0595 | main | Archivos necesarios para la API |
| Oct 10, 2026, 1:01 PM | @Jacquelin e0595 | main | Eliminación de algunos errores añadiendo lineas faltantes |
| Oct 10, 2026, 7:27 PM | @Jacquelin e0595 | main | Corrección de errores: código duplicado y errores logicos |
| Oct 10, 2026, 8:27 PM | @Jacquelin e0595 | main | Implementacion de un diseño colorido a las cartas de cambio de color |
| Oct 10, 2026, 9:52 PM | @Jacquelin e0595 | main | Eliminación de un detalle |
| Oct 10, 2026, 10:20 PM | @Jacquelin e0595 | main | Mensaje para tito |
| Oct 11, 2026, 00:51 AM | Axtraid | main | Resuelve conflictos al integrar el main |
| Oct 11, 2026, 00:56 AM | Axtraid | main | Deja de rastrear archivos generados al compilar |
| Oct 11, 2026, 20:14:27 | @Jacquelin e0595 | main | Documentacion del juego |
| Nov 3, 2026, 22:17:50 2026 | ZABNOOB | main | Se eliminaron errores en la reparticion de cartas, se redujo el espacio entre cartas, se cumplen las reglas cuando sale |

| | | | |
|---|---|---|---|
| | | | la primera carta al mazo, y se comprueba que sean 108 las cartas disponibles |
| Nov 4, 2026, 18:42:40 | @jandromt1 | main | Agrega servidor multihilo en C# y adaptacion de Form2 para sockets |
| Nov 4, 2026, 18:59:48 | @jandromt1 | main | Merge pull request #4 from Jacqueline0595/feature/sockets-uno |
| Nov 8, 2026, 18:32:04 | @Jacqueline0595 | main | Cambio de imagen |
| Nov 8, 2026, 20:24:52 | Axtraid | main | Fondo rojo nuevo y cada jugador se ve abajo en modo red |

# 5. API

Para conectar el juego UNO a una base de datos local, se desarrolló una API con Python y el framework FastAPI. Su función principal es recibir solicitudes de la aplicación cliente, procesar las operaciones del juego correspondientes, consultar o modificar la información almacenada en MySQL y devolver una respuesta en formato JSON. La API actúa como intermediaria entre la interfaz gráfica y la base de datos.

1. El cliente en C# se encarga de la interfaz gráfica y de enviar solicitudes a la API mediante la clase UnoApi.
2. La API en Python utiliza FastAPI para definir los endpoints, validar los parámetros, ejecutar las reglas del juego y coordinar el acceso a la base de datos.
3. Base de datos en MySQL, almacena los usuarios, partidas, participantes, cartas y requisitos de las acciones realizadas durante el juego.

Los principales endpoints de GET son los siguientes:

| Endpoint | Funcion |
|---|---|
| GET / | Comprueba que la API esté activa y devuelve un mensaje con su versión. |
| GET /hola | Devuelve los parámetros recibidos como ejemplo de su funcionamiento. |

| | |
|---|---|
| GET /usuarios | Consulta los usuarios registrados y los devuelve ordenados por su identificador. |
| GET /partidas | Obtiene una lista de partidas, incluyendo su estado y, cuando corresponda, el nombre del ganador. |
| GET /partidas/{partida_id} | Consulta el estado de una partida específica. |
| GET /partidas/{partida_id}/cartas | Consulta las cartas de una partida según su zona. |
| GET /partidas/{partida_id}/jugadas | Recupera el registro de jugadas de una partida almacenadas. |
| GET /partidas/{partida_id}/eventos | Consulta los eventos registrados. |

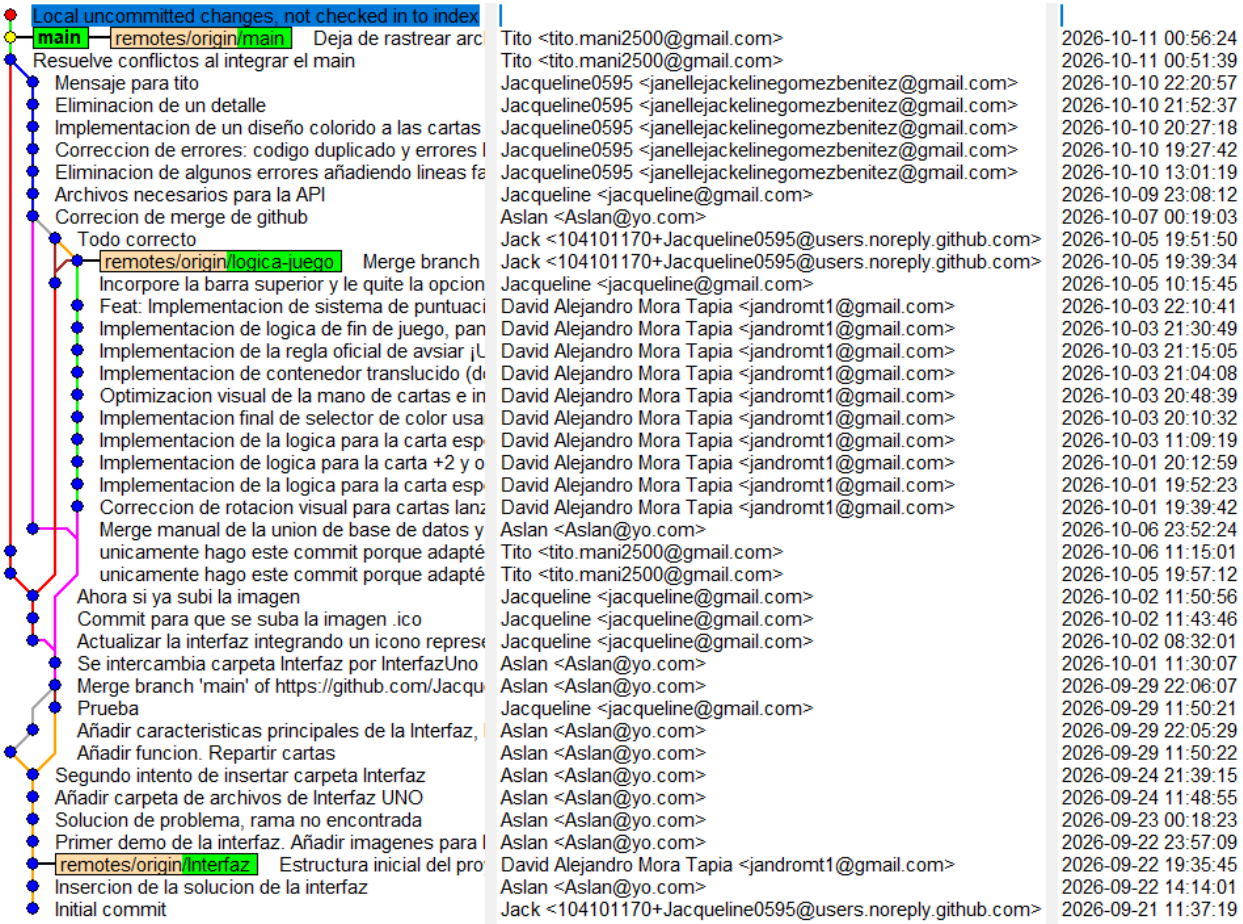
Los principales endpoints POST son los siguientes:

| Endpoint | Funcion |
|---|---|
| POST /usuarios | Registra un nuevo usuario, recibe su nombre y devuelve el identificador asignado. |
| POST /partidas | Crea una partida nueva a partir de los identificadores de los jugadores participantes. |
| POST /partidas/{partida_id}/robar | Permite al jugador del turno robar una carta del mazo. |
| POST /partidas/{partida_id}/jugar | Permite jugar una carta de la mano. |
| POST /partidas/{partida_id}/pasar | Permite pasar el turno después de robar una carta. |
| POST /partidas/{partida_id}/abandonar | Finaliza una partida en curso sin asignar un ganador. |

# 6. Estructura de Ramas Generadas (Branching Strategy)

Para evitar sobreescrituras accidentales en el código principal, se trabajó bajo una estrategia de ramas en GitHub donde cada funcionalidad se aisló antes del proceso de integración (`git merge`):

- **main**
  - **Interfaz**
  - **feature/logica-e-interfaz**
  - **feature/mejorar-baraja**
  - **jacque**
  - **logica-juego**
  - **tito**



# 7. Problemas Encontrados y Soluciones Aplicadas

Durante la ejecución del proyecto se presentaron desafíos técnicos tanto en la arquitectura de software como en el manejo del control de versiones en equipo, los cuales se resolvieron formalmente:

- **Problema 1: Conflictos de merge en Git debido a carpetas temporales de Visual Studio:** Al integrar el trabajo de diferentes integrantes, surgieron conflictos entre las versiones de los formularios. Se incluyen archivos generados por Visual Studio. Se revisaron los cambios antes de integrarlos, se resolvieron conflictos en la rama principal y se ajustó el archivo .gitignore.
- **Problema 2: Código duplicado y errores de lógica al combinar versiones de Form2:** Al unificar cambios en la interfaz, se detectaron líneas duplicadas y partes faltantes que provocaban errores de compilación o de ejecución. Después de revisar el Form2, se eliminaron las duplicaciones y se añadieron las líneas faltantes.
- **Problema 3: Problemas con la API:** Fue un reto poder conectar la api con la base de datos de mySQL, el investigar las credenciales necesarias y comandos para una correcta conexión fue fundamental para poder avanzar. Una vez investigado, surgen problemas con respecto a las funciones para recibir datos desde la URL, entre estos: Parametros o formatos incorrectos, errores de permisos para modificar la base de datos y problemas en comandos para conseguir la información. Después de revisar cada uno de estos problemas se pudieron arreglar dichos errores para su correcto funcionamiento.
- **Problema 4:** Notamos que cada vez que alguien compilaba el programa en C#, se subían automáticamente a GitHub carpetas pesadas como `bin/` y `obj/`. Esto ensuciaba el repositorio, hacía que las descargas fueran lentas y provocaba errores raros en nuestras computadoras porque cada máquina genera sus propios binarios. **Solución:** Usé el comando `git rm --cached` para quitar esos archivos del rastro de Git sin borrarlos de mi computadora, y luego configuré correctamente el archivo `.gitignore` para asegurarnos de que Visual Studio nunca más volviera a intentar subir esas carpetas de compilación al servidor.
- **Problema 5:** Cuando un jugador tenía que comer cartas del mazo, nos dimos cuenta de un error visual: las imágenes de las cartas nuevas se encimaban exactamente una arriba de la otra o quedaban ocultas en el fondo. El tamaño inicial de los archivos de imagen era tan grande que si un jugador juntaba muchas cartas, simplemente ya no cabían en la pantalla. Esto rompía el juego porque el usuario no tenía ninguna retroalimentación visual; comía cartas pero su mano parecía exactamente igual y no podía ver qué cartas nuevas le habían tocado. **Solución:** Para resolverlo, redimensionamos por código el tamaño original de las imágenes de las cartas para hacerlas más pequeñas y manejables. Además, programamos un algoritmo de reorganización automática (estilo abanico). Ahora, cada vez que un jugador toma una carta, el programa recalcula el espacio disponible en su zona de la pantalla, reduce los márgenes de separación y redistribuye todas las cartas en tiempo real. Así, sin importar cuántas cartas tenga un jugador en la mano, siempre se reacomodan visualmente y el usuario tiene la retroalimentación visual de que su mano creció.